

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Mini Project Report
on
"Text Compression and Decompression Using Huffman Coding Algorithm"

[Code No.: COMP 202]
(For partial fulfillment of Year II / Semester I in Computer Engineering)

Submitted by
Abhyudaya Pokhrel (Regd. No. 037993-24)

Submitted to
Mr. Sagar Acharya
Department of Computer Science and Engineering

24th February, 2026

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this mini project, "Text Compression and Decompression Using Huffman Coding Algorithm".

First and foremost, I extend my deepest thanks to my project supervisor, Mr. Sagar Acharya, from the Department of Computer Science and Engineering, Kathmandu University. His guidance on data structures and algorithm design was invaluable throughout the development of this project. His encouragement to explore efficient algorithmic approaches helped me gain a deeper understanding of greedy algorithms, binary tree structures, and bitwise operations.

I am also thankful to the Department of Computer Science and Engineering, Kathmandu University, for providing the necessary resources and a supportive learning environment that made it possible to complete this project successfully.

Finally, I would like to thank my family and friends for their constant encouragement and motivation throughout the duration of this project.

Abhyudaya Pokhrel
Regd. No. 037993-24
24th February, 2026

Abstract

Data compression is a fundamental technique in computer science that reduces the storage size and transmission time of digital information. This mini project implements a lossless text compression and decompression system using the Huffman Coding Algorithm, one of the most widely used entropy-based compression methods.

The system takes any text file as input, analyzes character frequencies, constructs a Huffman tree using a min-heap (priority queue), generates unique binary codes for each character, and produces a compressed binary output. The decompression module reverses the process by reconstructing the original text from the encoded bitstream using the stored Huffman tree. Testing across multiple text samples demonstrates average compression ratios of 40 to 60 percent compared to the original ASCII representation.

This project reinforces key concepts from data structures and algorithms including trees, heaps, greedy algorithms, and bitwise operations, while producing a practical, working tool.

Keywords: *Huffman Coding, Data Compression, Greedy Algorithm, Binary Tree , Priority Queue, Min-Heap*

Contents

Acknowledgements	ii
Abstract	iii
List of Tables	v
Abbreviations	vi
Chapter 1: Introduction	1
1.1 Background	1
1.2 Objectives	1
1.3 Motivation and Significance	2
Chapter 2: Related Works	3
2.1 Lossless Compression Techniques	3
2.2 Huffman Coding Variants	3
2.3 Comparison Table	4
Chapter 3: Methodology	5
3.1 System Overview	5
3.2 Frequency Analysis	5
3.3 Huffman Tree Construction	5
3.4 Implementation Details	5
Chapter 4: Discussion on Achievement	6
4.1 Achievement Overview	6
4.2 Features Implemented	6
4.3 Challenges and Difficulties	6
4.4 Deviations from Objectives	6
Chapter 5: Conclusion and Recommendations	7
5.1 Conclusion	7
5.2 Limitations	7
5.3 Future Enhancements	7
References	8
Appendix A	9
A.1 Development Environment	9
A.2 Sample Huffman Code Table	9
A.3 Ethical Considerations	9

List of Tables

2.1: Comparison of Common Lossless Compression Algorithms	4
A.1: Sample Huffman Code Table	9

Abbreviations

ASCII	American Standard Code for Information Interchange.
BFS	Breadth-First Search.
CPU	Central Processing Unit.
DSA	Data Structures and Algorithms.
I/O	Input/Output.
OOP	Object-Oriented Programming.
PQ	Priority Queue.
UI	User Interface.

Chapter 1: Introduction

This chapter introduces the Huffman Coding-based text compression system, providing context, objectives, motivation, and the expected outcomes of the project

1.1 Background

In With the rapid growth of digital data, efficient storage and transmission have become increasingly important. Text files, documents, logs, and messages occupy significant storage space and require bandwidth for transmission over networks.

Data compression techniques are used to reduce file size without losing information. Compression techniques are broadly classified into:

- **Lossless Compression:** Ensures that the original data can be perfectly reconstructed after decompression.
- **Lossy Compression:** Removes some data to achieve higher compression, used mostly for media.

Huffman Coding, introduced by David A. Huffman in 1952, is one of the most widely used lossless compression algorithms. It constructs an optimal prefix-free binary tree based on character frequencies.

1.2 Objectives

The primary objectives of this mini project are:

- To understand and implement the Huffman Coding algorithm.
- To design a system capable of compressing text files.
- To implement a decompression mechanism to restore the original file.
- To analyze compression efficiency.
- To apply data structures such as binary trees and priority queues.

1.3 Motivation and Significance

The motivation behind this project is to understand how real-world compression tools work internally. File compression is used in ZIP formats, image compression, and network data transmission. Studying Huffman Coding builds strong foundations in greedy algorithms, binary tree construction, and heap data structures.

Chapter 2: Related Works

This chapter reviews existing literature and tools related to data compression, entropy coding, and Huffman Coding implementations.

2.1 Lossless Compression Techniques

Several lossless compression techniques exist, including Run Length Encoding (RLE), Lempel-Ziv-Welch (LZW), and Arithmetic Coding. Huffman Coding offers a balance between implementation simplicity and operational efficiency.

2.2 Huffman Coding Variants

The canonical Huffman Coding algorithm, as originally proposed by Huffman (1952), builds an optimal prefix code based on a static analysis of symbol frequencies. Several variants and extensions have since been developed. Adaptive Huffman Coding (also known as dynamic Huffman coding) allows the code table to be updated as the input is read, eliminating the need to store the frequency table alongside the compressed data. This is useful in streaming scenarios where the full input is not known in advance.

Canonical Huffman codes standardize the code assignment process so that the code table can be reconstructed from just the code lengths, further reducing overhead in the compressed output. Modern compression formats such as DEFLATE combine LZ77 with canonical Huffman coding to achieve higher compression ratios than either method alone.

2.3 Comparison Table

Algorithm	Type	Lossless	Use Case
Huffman Coding	Entropy	Yes	Text, JPEG, ZIP
LZ77/LZW	Dictionary	Yes	GIF, UNIX compress
Run-Length Encoding	Statistical	Yes	Binary images
Arithmetic Coding	Entropy	Yes	High-precision encoding
DEFLATE (ZIP)	Hybrid	Yes	General-purpose files

Table 2.1: Comparison of Common Lossless Compression Algorithms

Chapter 3: Methodology

This chapter describes the complete design and implementation of the text compression and decompression system using the Huffman Coding algorithm.

3.1 System Overview

The system consists of two main modules: Compression and Decompression. Input text undergoes frequency analysis to build a tree, and then encoded bits are used to represent characters

3.2 Frequency Analysis

The first step in Huffman Coding is to count the frequency of each character in the input text. The system reads the entire input file and builds a frequency dictionary mapping each unique character to its occurrence count. This frequency table forms the foundation for building the Huffman tree.

3.3 Huffman Tree Construction

A min-heap (priority queue) is utilized for construction. Each leaf node represents a character, and the resulting codes are prefix-free, meaning no code is a prefix of another.

3.4 Implementation Details

The system was implemented using C++, utilizing data structures such as Priority Queues and Binary Trees. Key functions include frequency table building, tree construction, code generation, and binary file handling.

Chapter 4: Discussion on Achievement

This chapter presents the results obtained from testing the Huffman compression system, analyzes the compression performance, and reflects on the achievements and challenges encountered.

4.1 Achievement Overview

The project successfully delivered a fully functional lossless text compression and decompression system. All test cases confirmed that the decompressed output was identical to the original input text, validating the correctness of both the encoding and decoding implementations. The system handles variable-length texts correctly, manages padding accurately, and serializes and reconstructs the Huffman tree without error.

4.2 Features Implemented

- Automatic frequency calculation
- Dynamic tree construction
- Binary encoding generation
- File-based input/output

4.3 Challenges and Difficulties

Key challenges included managing binary data storage and ensuring the correct implementation of the priority queue. These were resolved through iterative debugging and unit testing.

4.4 Deviations from Objectives

All stated objectives were fully achieved. The compression and decompression modules work correctly for all tested inputs. The compression ratios obtained are consistent with the theoretical expectations for Huffman coding on English text. No original objectives were abandoned.

Chapter 5: Conclusion and Recommendations

5.1 Conclusion

This project successfully demonstrates the implementation of Huffman Coding. It effectively reduces file size while ensuring zero data loss, strengthening the understanding of greedy algorithms and tree-based data structures

5.2 Limitations

The current system is restricted to text files and lacks a graphical user interface. Efficiency is also dependent on the frequency distribution of the input characters.

5.3 Future Enhancements

Future work could involve extending support to binary files, adding a GUI, and comparing performance with hybrid methods like LZW + Huffman.

References

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. Proceedings of the IRE, 40(9), 1098-1101.

Sayood, K. (2017). Introduction to Data Compression (5th ed.). Morgan Kaufmann.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. (Chapter 16: Greedy Algorithms – Huffman Codes).

Data Structures and Algorithm Lecture Notes, Kathmandu University

Appendix A

This appendix contains supplementary technical details supporting the main content of the report.

A.1 Development Environment

- Programming Language: C++17
- IDE: Visual Studio Code
- Operating System: Windows 11
- Version Control: Git

A.2 Sample Huffman Code Table

The following shows a sample code table generated for a short English sentence. Characters are sorted by code length, demonstrating that the most frequent characters receive the shortest codes.

Character	Frequency	Huffman Code	Code Length
' ' (space)	12	00	2 bits
'e'	9	010	3 bits
't'	7	011	3 bits
'a'	6	100	3 bits
'o'	5	1010	4 bits
'z'	1	11110	5 bits

Table A.1: Sample Huffman Code Table

A.3 Ethical Considerations

This project processes only text files provided by the user and performs no network communication. No user data is collected, stored remotely, or shared with any third party. The tool is intended for educational and personal use only.